

ArchSync: Evidence-Backed Conformance Checking for TypeScript Service Architectures

ANONYMOUS AUTHOR(S)

Passing tests do not establish that a service change conforms to its intended architecture. ArchSync links a machine-readable architectural contract to selected HTTP, PostgreSQL, Redis and AMQP observations in TypeScript, retains source evidence, and maps introduced findings to PASS, BLOCK or REVIEW at a Git change. We evaluate this deterministic prototype on co-developed datasets comprising 20 architecture patches and 40 annotated detector signals; incremental replay reuses the same patches. A separate reliability campaign checks 786,432 finite rule-graph combinations against a matrix oracle and repairs adversarial extraction and cache faults. All 20 patch classifications agree with their development labels, and the latest implementation passes 315 Guardian regression tests. These results support contract coherence and regression reliability, not representative accuracy, comparative superiority or developer productivity. The contribution is an auditable integration of service-level observations, explicit rules and change-time decisions, with retained failures and bounded evidence for its behavior.

CCS Concepts: • **Software and its engineering** → **Software architecture**; *Software verification*.

Additional Key Words and Phrases: architecture conformance, service-oriented systems, static analysis, architecture drift, reproducibility

1 INTRODUCTION

Service-oriented systems can preserve functional behavior while introducing a forbidden dependency, losing a required interaction, or adding infrastructure that needs a design decision. These changes concern different architectural judgments. A useful check must connect an observed interaction to the declared contract, identify supporting source evidence, and distinguish a violation from an evolution. ArchSync addresses this information-extraction and decision-support problem with deterministic rules; no learned model or AI effectiveness claim is required.

Our contribution is an executable integration: (1) a typed expected/observed service graph with explicit constraints; (2) provenance-aware extraction of selected TypeScript interactions and evidence; and (3) a Git-diff gate that reports introduced violations separately from evolution. Model comparison and dependency constraints are established ideas. Our empirical contribution is a bounded, reproducible verification of this integration, including failure-first hardening rather than a claim of new general-purpose program analysis.

We ask whether the prototype reconstructs declared graph changes and detector signals (RQ1), classifies controlled changes correctly (RQ2), supplies the declared evidence locations (RQ3), and preserves results under replay and incremental execution (RQ4). These are development-contract questions, not estimates of unseen-project accuracy.

2 BACKGROUND AND RELATED WORK

This scoped narrative synthesis establishes context for ArchSync’s design; it is not a systematic literature review. Sources were selected purposively from the retained keyword-search corpus for architecture conformance, drift detection, and static-analysis reliability, emphasizing 2022–2025. Four earlier works supply concept origins. Selection and interpretation may omit relevant approaches; coverage completeness and absence-of-prior-work claims are outside scope.

Erosion and reconstruction. A mapping of architecture erosion describes its manifestations and consequences [13]. Code-review studies identify symptoms in OpenStack and later OpenStack/Qt projects [12, 14]; their shared project contexts are not independent replications. Practitioner evidence identifies documentation, responsibilities, and shared practices as ways to manage drift [3]. Reconstruction taxonomies organize recovery strategies [7, 19], while a microservice mapping compares tool purposes and availability [4]. View-based analysis compares intended and reconstructed specifications in an industrial case [23]. ArchSync restricts its observation boundary to explicit service and infrastructure relationships.

Conformance. Reflexion Models, static conformance comparisons, and DCL provide the origins of model comparison and allowed/forbidden dependency rules [10, 17, 22]; neither concept is new here. CALint applies dependency-rule checks to Python [6]. ArchLintor is a close TypeScript comparison, detecting module-interaction violations and supporting CI/CD [20]. ArchSync’s typed service topology differs from that module boundary; no head-to-head accuracy result is claimed. IaC metrics address deployment coupling [18], and 3Erefactor searches for executable inconsistency-reducing repairs [15]. LLM-generated architectural tests distinguish execution success from correct rule expression [16]. ArchSync executes explicit rules deterministically, but that choice alone implies neither accuracy nor superiority.

Change-time feedback. Commit-level prediction studies architectural change [25]; repository mining tracks microservice evolution [8]; industrial conformance visualization supplies practitioner evidence [24]. A predicted or observed change need not violate intended architecture. ArchSync instead reports findings introduced at a Git head, preserving separate violation and evolution decisions.

Evaluation reliability. Replication support remains an evaluation concern [11]. Decomposition surveys cover an adjacent problem [1], whereas a microservice-recovery comparison executes nine tools and reports individual and combined recovery performance [21]. That study supplies an external-comparison precedent, not scores directly comparable with ArchSync. Static visual models and multiple dependency models motivate future multi-source work [2, 5]. Interrogation testing exposes both precision and soundness failures [9], motivating our explicit hard negatives. These sources establish comparison dimensions rather than exhaustive novelty or representative accuracy.

3 EXECUTABLE CONTRACT AND CHANGE-TIME ANALYSIS

Let $M = (V, E, R)$ be the expected architecture, with components V , directed typed relationships E , and rules R . The observed graph O is reconstructed from supported source constructs. Component identifiers and metadata from M condition reconstruction; this is not architecture recovery without prior structure. Edges collapse repeated observations with the same source, target and kind, while retaining their file/line evidence.

A deny rule rejects matching edges. An allow rule restricts destinations of the selected outgoing relations. A require rule demands a matching direct edge for each selected source; require-path demands a nonempty directed path, optionally restricted to one edge type. An observed rule violation takes classification precedence; otherwise a graph difference is evolution, and an unchanged graph is no-impact. The analyzer never rewrites the expected model.

The TypeScript adapter recognizes selected global fetch calls and imported PostgreSQL, Redis and AMQP client operations. Import provenance distinguishes package clients from method-name lookalikes. HTTP targets use host identity rather than port/path identity; AMQP broker reachability does not demonstrate routing compatibility. Unsupported wrappers, unresolved dynamic endpoints and inter-file flows can remain invisible. Evidence scores are heuristics, not calibrated probabilities.

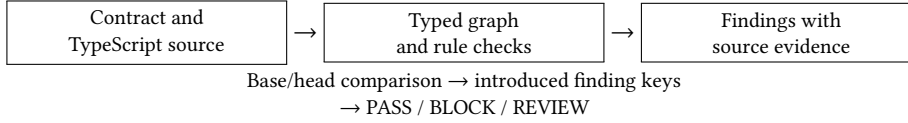


Fig. 1. Source observations and explicit contracts feed a change-time decision. The model remains under separate human control.

For a base/head pair, the gate reconstructs the base, analyzes affected source components and merges retained observations with new ones. Full-head scanning supplies a consistency oracle. New violation keys produce BLOCK, new evolution keys produce REVIEW, and no introduced finding produces PASS. Existing findings remain visible. Keys operate at relationship level, so an additional offending call on an existing edge need not introduce a new key. PASS therefore means no newly recognized gated finding, not complete architectural safety. REVIEW requires a separate architecture decision; unchanged inputs remain REVIEW after a human rejection. Repository governance is not implemented by the classification alone.

The expected model is an input contract, not ground truth inferred from the emitted graph. Call-site evidence points to observed code. A missing required relation has no absent call to localize, so it receives a deterministic component anchor. These evidence types are scored separately. Cache identity binds the baseline and configuration; later hardening also separates repository scope and analyzer version and checks payload integrity. A checksum protects against accidental corruption, not a writer able to replace both payload and checksum.

Worked controlled example. In D1 case 06, rule ARCH-001 forbids a direct relationship from frontend to payment-service. The patch adds `payDirectly` and a `fetch` call to the payment endpoint. Reconstruction adds one HTTP edge between those components; the deny rule produces an error finding with evidence at `frontend/src/app.ts`, line 14 of the patched file. Because the finding is introduced relative to the base, the Git gate returns BLOCK. This is the frozen case’s declared behavior, not a new production observation. A new relation without a violated rule instead needs REVIEW; the tool does not infer that the architectural change is approved.

4 HISTORICAL CONTROLLED EVALUATION

Objects and independence. D1 is a synthetic Order Platform baseline with five components and five relationships. Its 20 isolated patches comprise nine no-impact changes, seven violations and four evolutions. Topology labels preceded integration, but executable patches and precise evidence locations co-evolved with the analyzer. D2 has four positive and four hard-negative signals for each of five detector groups: `fetch`, `PostgreSQL`, `Redis`, `AMQP publish` and `AMQP consume`. It was constructed from v0.1 weaknesses and informed v0.2. Neither dataset is a holdout. P3 replays D1 through Git; it is not a third independent dataset.

Procedure and units. D1 validates hashes, patch applicability and expected changes, then runs the baseline and each isolated patch twice (42 executions). D2 runs both pinned versions twice (four executions). P3 creates fresh baseline repositories, applies one patch, performs cold and warm checks, and separately scans the full head. Results are compared after excluding elapsed time and cache-hit state. The 20 cold/warm pairs plus full scans require 80 analyzer calls. D1 items are graph nodes/edges; D2 items are annotated detector–edge–file–line signals, which can share graph edges. Evidence matches are case-level, not recall over all call sites.

Results. Table 1 reports the meaningful changed-item and case denominators. Full-graph totals also matched 105 nodes and 108 edges, but repeatedly include baseline structure. On D2, v0.1 yielded TP=18, FP=4, FN=2 and TN=16; v0.2 yielded TP=20, FP=0, FN=0 and TN=20. The four false positives were local PostgreSQL/AMQP-shaped receivers;

Table 1. Historical development-contract results; repeated baseline structure is not independent evidence.

Measure	Agreement/count
D1 changed nodes / edges	5/5; 12/12
D1 classifications / violation rule sets	20/20; 7/7
Call-site cases / missing-relation anchors	9/9; 2/2
P3 decisions / exact deltas	20/20; 20/20
P3 incremental/full scan / cold-warm replay	20/20; 20/20
P3 parsed / head file instances	57/189

the two misses were Redis operations. These targeted repairs are within-tool regression evidence, not external comparative improvement. Duplicate normalized output matched, and P3 cache miss/hit results agreed in all 20 cases. Incremental/full agreement can preserve the same semantic omission in both modes.

The original Windows recording used Node.js 22.16.0 and an Intel Core i7-13650HX. Cold/warm nearest-rank p50 values were 518.51/242.62 ms and p95 values 531.05/249.30 ms. They combine parsing, Git, cache I/O and reporting; RAM, power/load and filesystem-cache state were unrecorded. These are descriptive observations, not causal speedup or scalability evidence. Historical functional replays passed Linux and Windows; current local hardening below does not constitute a new cross-platform study.

5 SEPARATE TECHNICAL RELIABILITY EXTENSION

The subsequent 2026-09-26 campaign targets local development defects and preserves original failures. It does not revise the preceding observations or their pinned analyzer packages.

Method provenance. OpenAI Codex with delegated coding agents assisted in constructing the finite oracle, adversarial inputs and mutation harness, modifying the implementation, executing local checks and inspecting outputs. Reported counts come from retained executable test reports, not generated estimates. Test refinement after inspecting failures is disclosed below; the checks are development evidence. Cross-agent inspection is automated code review, not independent human annotation or adjudication. No human validation signoff is asserted for this extension.

Finite rule checking. A separately implemented adjacency-matrix and transitive-closure oracle checks all 4,096 directed graphs on three fixed nodes, two edge types and 12 non-self edge slots. Each graph is evaluated against 192 combinations of deny/allow/require/require-path, exact/wildcard selectors and type constraints: 786,432 rule-graph evaluations. These are finite combinations, not independent samples. Additional checks apply ordering and consistent renaming on 241 graph states and inverse addition/removal deltas on all 4,096. Self-loop inputs, larger graphs, arbitrary wildcard fragments, extraction and component-metadata changes are outside this domain.

Test sensitivity. Eight declared implementation mutations target rule enforcement, path traversal and classification. The initial suite killed seven; violation demotion survived because classification was not asserted. Adding the missing assertion produced eight kills, no survivors and no execution errors in the final audit. Both rounds are retained. This is test-development evidence against a small fault model, not a whole-program mutation score or independent validation.

Extraction and incremental faults. Adversarial source tests exposed type-only imports treated as runtime clients, protocol mismatches, shadowed bindings and mistaken URL expressions. Binding symbols per file separates local names from package/global names. Later checks exposed wrong-host string concatenation and stale provenance after direct reassignment. Static string composition now resolves the complete URL. A further 14-case extension reproduced 11 failures caused by destructuring, unary updates and loop writes; all 43 cumulative adversarial cases pass after

repair. Recognized writes conservatively invalidate the binding throughout its file, including earlier legitimate uses. Unresolved composition emits no relationship. This is not a sound, flow-sensitive analysis.

Incremental checks exposed missed unusual/ignored filenames, nested-component evidence loss, cross-project cache reuse and corrupt-cache behavior. Exact source ownership, Git path handling, repository/version-bound cache identity and payload checks address the reproduced cases. The extension records failing inputs, repaired outputs and remaining limitations separately. Its full local suites passed 126 Core and 315 Guardian tests on Node.js 22.16.0/macOS arm64; those totals describe regression execution only. Two inert compiler-host callbacks are excluded from Guardian’s configured coverage. The current analyzer is unreleased semantics version 0.4; its passing reuse of all 20 D1 cases is still development evidence.

6 VALIDITY AND IMPLICATIONS

Construct and internal validity. A typed structural view does not measure responsibilities, runtime quality or usefulness. Co-developed labels and implementation share assumptions. A separately coded oracle may still share a mistaken specification. D2 specificity covers explicit hard negatives, not all possible nonedges. File/line agreement measures one declared case location, not evidence completeness or ranking. Perfect replay can reproduce an incorrect inference.

External and comparative validity. One synthetic system and targeted fixtures cannot establish representative TypeScript accuracy. Dynamic templates, alternative fallbacks, property mutation, imported-constructor rebinding, wrappers and inter-file flows remain limitations. No accepted external-tool comparison or independently labeled real-repository holdout exists. Module-import dependencies and service calls require a nonempty common abstraction before comparison; unsupported relations must not become comparator false negatives. Narrative source selection can omit relevant work and does not establish exhaustive novelty. Missing full texts remain identified in the review register; no broader review is implied.

Conclusion validity. The small changed-item counts and finite graph enumeration do not justify population confidence intervals or significance tests. Coverage and mutant kills measure tested implementation behavior, not production accuracy. The next study must freeze repository selection and tools before predictions, label the entire declared source scope (including files without predictions), retain annotation disagreements, and report per-repository errors, supported-vocabulary coverage and failures. A paired, controlled workload study is needed for performance; practitioner observations are needed for false-block burden and review effort.

7 CONCLUSION

ArchSync combines service-level extraction, explicit conformance rules and evidence-backed Git decisions. Controlled datasets establish agreement with development contracts; the separate hardening campaign finds and repairs concrete reliability faults while exposing residual limitations. The strongest supported result is auditable engineering coherence. Independent repositories, compatible baselines and practitioner evaluation remain necessary before broader accuracy or usefulness claims.

REFERENCES

- [1] Yalemisew Abgaz, Andrew McCarren, Peter Elger, David Solan, Neil Lapuz, Marin Bivol, Glenn Jackson, Murat Yilmaz, Jim Buckley, and Paul Clarke. 2023. Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review. *IEEE Transactions on Software Engineering* 49, 8 (2023), 4213–4242. <https://doi.org/10.1109/TSE.2023.3287297>
- [2] Metin Altunışık, Hasan Sözer, and Gonca Gürsun. 2024. Software Architecture Recovery from Multiple Dependency Models. In *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*. ACM, 1185–1192. <https://doi.org/10.1145/3605098.3635917>

- [3] Emilie Anthony, Astrid Berntsson, Tiziano Santilli, and Rebekka Wohlrab. 2024. We're Drifting Apart: Architectural Drift from the Developers' Perspective. In *2024 IEEE 21st International Conference on Software Architecture*. IEEE, Hyderabad, India, 101–111. <https://doi.org/10.1109/ICSA59870.2024.00018>
- [4] Alexander Bakhtin, Xiaozhou Li, Jacopo Soldani, Antonio Brogi, Tomas Cerny, and Davide Taibi. 2024. *Tools Reconstructing Microservice Architecture: A Systematic Mapping Study*. Springer Nature Switzerland, 3–18. https://doi.org/10.1007/978-3-031-66326-0_1
- [5] Tomas Cerny, Amr S. Abdelfattah, Jorge Yero, and Davide Taibi. 2024. From static code analysis to visual models of microservice architecture. *Cluster Computing* 27, 4 (April 2024), 4145–4170. <https://doi.org/10.1007/s10586-024-04394-7>
- [6] Clelio Orlando de Oliveira Junior, Jonathan Carvalho, Fábio Fagundes Silveira, Tiago Silva da Silva, and Eduardo Martins Guerra. 2022. CALint: A Tool for Enforcing the Clean Architecture's Dependency Rule in Python. In *Computational Science and Its Applications – ICCSA 2022 Workshops: Malaga, Spain, July 4–7, 2022, Proceedings, Part V*. Springer International Publishing, 534–549. https://doi.org/10.1007/978-3-031-10548-7_39
- [7] Stéphane Ducasse and Damien Pollet. 2009. Software Architecture Reconstruction: A Process-Oriented Taxonomy. *IEEE Transactions on Software Engineering* 35, 4 (2009), 573–591. <https://doi.org/10.1109/TSE.2009.19>
- [8] Patric Genfer and Uwe Zdun. 2024. Exploring Architectural Evolution in Microservice Systems Using Repository Mining Techniques and Static Code Analysis. In *Software Architecture: 18th European Conference, ECSA 2024, Luxembourg City, Luxembourg, September 3–6, 2024, Proceedings*. Springer Nature Switzerland, 157–173. https://doi.org/10.1007/978-3-031-70797-1_10
- [9] David Kaindlstorfer, Anastasia Isychev, Valentin Wüstholtz, and Maria Christakis. 2024. Interrogation Testing of Program Analyzers for Soundness and Precision Issues. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, New York, NY, USA, 319–330. <https://doi.org/10.1145/3691620.3695034>
- [10] Jens Knodel and Daniel Popescu. 2007. A Comparison of Static Architecture Compliance Checking Approaches. In *Sixth Working IEEE/IFIP Conference on Software Architecture*. IEEE, Los Alamitos, CA, USA, 12–21. <https://doi.org/10.1109/WICSA.2007.1>
- [11] Marco Konersmann, Angelika Kaplan, Thomas Kühn, Robert Heinrich, Anne Koziol, Ralf H. Reussner, Jan Jürjens, Mahmood al Doori, Nicolas Boltz, Marco Ehl, Dominik Fuchß, Katharina Großer, Sebastian Hahner, Jan Keim, Matthias Lohr, Timur Sağlam, Sophie Schulz, and Jan-Philipp Töberg. 2022. Evaluation Methods and Replicability of Software Architecture Research Objects. In *2022 IEEE 19th International Conference on Software Architecture (ICSA)*. IEEE, Honolulu, HI, USA, 157–168. <https://doi.org/10.1109/ICSA53651.2022.00023>
- [12] Ruiyin Li, Peng Liang, and Paris Avgeriou. 2023. Warnings: Violation symptoms indicating architecture erosion. *Information and Software Technology* 164 (2023), 107319. <https://doi.org/10.1016/j.infsof.2023.107319>
- [13] Ruiyin Li, Peng Liang, Mohamed Soliman, and Paris Avgeriou. 2022. Understanding Software Architecture Erosion: A Systematic Mapping Study. *Journal of Software: Evolution and Process* 34, 3 (2022), e2423. <https://doi.org/10.1002/smr.2423>
- [14] Ruiyin Li, Mohamed Soliman, Peng Liang, and Paris Avgeriou. 2022. Symptoms of Architecture Erosion in Code Reviews: A Study of Two OpenStack Projects. In *2022 IEEE 19th International Conference on Software Architecture (ICSA)*. IEEE, 24–35. <https://doi.org/10.1109/icsa53651.2022.00011>
- [15] Jingwen Liu, Wuxia Jin, Junhui Zhou, Qiong Feng, Ming Fan, Haijun Wang, and Ting Liu. 2024. 3Erefactor: Effective, Efficient and Executable Refactoring Recommendation for Software Architectural Consistency. *IEEE Transactions on Software Engineering* 50, 10 (Oct. 2024), 2633–2655. <https://doi.org/10.1109/tse.2024.3449564>
- [16] Andrielly Lucena, Everton L. G. Alves, and João Brunet. 2025. Language Models as Architectural Gatekeepers: Automating Conformance Checking from Natural Language. In *Anais do XXXIX Simpósio Brasileiro de Engenharia de Software (SBES 2025)*. Sociedade Brasileira de Computação, 713–719. <https://doi.org/10.5753/sbes.2025.11100>
- [17] Gail C. Murphy, David Notkin, and Kevin J. Sullivan. 1995. Software Reflexion Models: Bridging the Gap Between Source and High-Level Models. In *Proceedings of the Third ACM SIGSOFT Symposium on Foundations of Software Engineering*. ACM, New York, NY, USA, 18–28. <https://doi.org/10.1145/222124.222136>
- [18] Evangelos Ntontos, Uwe Zdun, Jacopo Soldani, and Antonio Brogi. 2022. *Assessing Architecture Conformance to Coupling-Related Infrastructure-as-Code Best Practices: Metrics and Case Studies*. Springer International Publishing, 101–116. https://doi.org/10.1007/978-3-031-16697-6_7
- [19] Abdul Qayum, Mengqi Zhang, Simon Colreavy, Muslim Chochlov, Jim Buckley, Dayi Lin, and Ashish Rajendra Sai. 2024. A Framework and Taxonomy for Characterizing the Applicability of Software Architecture Recovery Approaches: A Tertiary-Mapping Study. *Software: Practice and Experience* (2024). <https://doi.org/10.1002/spe.3364> First published online 17 July 2024.
- [20] Matheus Resende, Rafael Durelli, and Ricardo Terra. 2025. ArchLintor: detecting architectural violations in TypeScript systems. In *Anais do XXXIX Simpósio Brasileiro de Engenharia de Software (SBES 2025)*. Sociedade Brasileira de Computação, 900–906. <https://doi.org/10.5753/sbes.2025.11075>
- [21] Simon Schneider, Alexander Bakhtin, Xiaozhou Li, Jacopo Soldani, Antonio Brogi, Tomas Cerny, Riccardo Scandariato, and Davide Taibi. 2025. Comparison of static analysis architecture recovery tools for microservice applications. *Empirical Software Engineering* 30, 5 (June 2025). <https://doi.org/10.1007/s10664-025-10686-2>
- [22] Ricardo Terra and Marco Tulio Valente. 2009. A Dependency Constraint Language to Manage Object-Oriented Software Architectures. *Software: Practice and Experience* 39, 12 (2009), 1073–1094. <https://doi.org/10.1002/spe.931>
- [23] Burak Uzun and Bedir Tekinerdogan. 2024. Detecting Deviations in the Code Using Architecture View-Based Drift Analysis. *Computer Standards & Interfaces* 87 (2024), 103774. <https://doi.org/10.1016/j.csi.2023.103774>
- [24] Filip Zamfirov, Andrei Radulescu, Jacob Krüger, and Michel R. V. Chaudron. 2025. Lessons from Visualizing Software Architecture Structure Conformance at Thermo Fisher Scientific. In *Software Engineering and Advanced Applications: 51st Euromicro Conference, SEAA 2025, Salerno, Italy,*

September 10–12, 2025, *Proceedings, Part III*. Springer Nature Switzerland, 372–389. https://doi.org/10.1007/978-3-032-04207-1_25
 [25] Wenjing Zhan, Ran Mo, Yingjie Jiang, and Dongyu Wang. 2025. Just-in-Time Prediction of Software Architectural Changes Through Commit-Level Analyses. *IEEE Transactions on Software Engineering* 51, 8 (Aug. 2025), 2285–2304. <https://doi.org/10.1109/tse.2025.3587849>

A REPRODUCTION AND INTERPRETATION DETAILS

This appendix supplies operational detail for the single-column review profile. It is supplied as a separate supplement with the compact profile; acceptance of supplementary uploads is not established by the public instructions. Its purpose is to make the development evidence inspectable, not to add independent observations.

A.1 Keep historical and current evidence separate

Historical D1/D2/P3 observations are reproduced from their pinned benchmark inputs and packaged analyzers. The reporting derivation uses benchmark commit 24d63ebf2fc3075a1d64f1eaff38cdc0b7f586fb. Its ground truth, Phase 2 and Phase 3 records have individual hashes. Rerunning a current analyzer on that same corpus answers a different question: whether the updated implementation preserves known development behavior. The latter cannot silently replace the former in a before/after table.

The artifact therefore distinguishes original packages, current source snapshots, current distribution builds, first failing runs, repaired runs and unchanged historical result inputs. Current source and package hashes identify the evaluated candidate; an unreleased version string alone does not identify its bytes. The accompanying manifest maps each new manuscript count to its raw receipt. A successful package installation is an operational compatibility check, not an independent assessment of architectural correctness.

The historical evidence-location decomposition is explicitly post-hoc. Nine finding-bearing cases concern call sites; cases 08 and 17 concern deterministic anchors for missing edge/path requirements. Case 15 contains two violated rules but counts once. A match in these finite cases cannot establish that all useful evidence was found, correctly ranked or understood by a developer.

A.2 Finite oracle construction

For three nodes, six ordered non-self node pairs and two edge types give 12 binary edge slots, hence $2^{12} = 4096$ graphs. For each graph, an adjacency matrix is built independently for all edges and for each individual edge type. Direct requirements query that matrix; path requirements query its transitive closure. The diagonal is not initially true: a required path must contain an edge. A cycle may nevertheless establish a nonempty path back to its starting node.

For each of four rule kinds, the source and target independently range over three exact identifiers and a full wildcard, and the edge-type restriction ranges over unrestricted and the two types. Thus $4 \times 4 \times 4 \times 3 = 192$ rule configurations are checked for every graph. The oracle compares per-rule violation counts, aggregate violations and classification. It imports no production graph-building, selector-matching or traversal helper. This structural separation reduces direct implementation reuse but does not establish independent authorship or correctness of the shared rule interpretation.

Ordering and consistent-renaming checks use masks 0 through 4080 in steps of 17, giving 241 graph states. Inverse-delta checks compare every graph with the empty-edge graph. They verify that added edges in one direction are removed edges in the other, and that edge changes without rules classify as evolution. These checks do not enumerate arbitrary pairs of graphs, metadata changes, arbitrary wildcard fragments or source-language programs. The chosen domain is exhaustive only within its declared bounds.

A.3 Mutation procedure and retained failures

The mutation harness copies the relevant source and test into a temporary isolated workspace, changes one declared expression at a time, and invokes the same test suite with a fixed timeout. Production source remains unchanged by this procedure. Every run retains the replacement text, original and mutated source hashes, test hash, command output, exit state and native report. A passing unmodified baseline is required before mutant outcomes are interpreted.

Mutant states distinguish test passes, assertion failures and execution errors. The eight faults disable deny enforcement, invert allow logic, invert direct-require satisfaction, ignore path edge types, prevent traversal beyond one edge, miss path target detection, demote violation, or demote evolution. The scope is intentionally narrow. A compile error or timeout is not silently counted as an assertion kill, and no claim about all equivalent mutants is made.

The original surviving classification mutation matters: it demonstrated that correct finding counts alone did not guarantee the test checked the returned decision. After the assertion was added, the same fault was detected. Reporting only the final perfect score would hide that development intervention. Both versions of the test evidence remain available, so reviewers can distinguish test strengthening from confirmation on an untouched sample.

A.4 Detector repairs and residual uncertainty

The first adversarial set contains 18 cases: 15 failed before intervention and all 18 passed after it. Positive controls check aliases, captured resources and an unshadowed global fetch alongside local bindings. Type-only import tests deliberately include source that a TypeScript compilation gate would reject. They test the analyzer’s robustness when parsing source without that gate, not the prevalence of such constructs in real projects.

A later intervention added eight cases for concatenation and reassignment: seven failed in the cumulative 26-case pre-repair suite. Three further boundary controls were added after repair, yielding 29 passing adversarial cases. These before/after suites therefore have different total sizes, and a naive percentage comparison would be misleading. An old positive fixture concatenated a suffix into an invalid URL port; the fixture was corrected to a valid path suffix and the invalid-port input retained as a negative regression.

A third intervention adds 14 cases to the 29-case suite: 12 write scenarios and two controls for unrelated or shadowed bindings. The identical 43-case suite has 11 failures and 32 passes before the change, then 43 passes afterward. One added write scenario already passed. The repair resolves variable identities within array/object destructuring (including rest/default targets), increment/decrement operations and assignment targets in for-in/for-of loops. For example, a URL binding overwritten by an array assignment must not retain its initializer as the target of a later fetch. These constructed programs test specified behavior; they are not independent observations of real-world prevalence.

Recognized-write invalidation is conservative and file-wide. It can suppress a genuine call that appears before re-assignment; it does not implement statement-order data flow. Property mutation, imported constructor rebinding, dynamically built templates and alternative fallback destinations remain separate risks. When no edge is emitted, the tool does not prove that no runtime relationship exists. Confidence constants attached to emitted evidence cannot resolve this missing-observation uncertainty. Analyzer version 0.4 invalidates caches from earlier semantics. The full repository gate passed from a clean temporary candidate whose 410 tracked files matched the working source byte-for-byte. The first full run stopped at a CLI assertion still expecting version 0.3; that failure is retained alongside the corrected assertion and passing rerun. These are packaging and regression checks, not independent reproduction.

A.5 Incremental and cache experiment boundary

The incremental suite creates temporary Git repositories and checks additions, modifications, deletions, renames within and between components, and complete-component deletion. Cold and warm results are compared with fresh full scans. The initial nine adversarial cases all failed on the original implementation; a later ignored-source case exposed a further false PASS. Before and after receipts preserve those observations.

Most differential cases compare classification, decisions, finding count and scanned-file count. Cold/warm comparisons additionally inspect architecture deltas and introduced/resolved findings. The nested-component case compares the complete observed graph and evidence. This distinction is important: aggregate count agreement alone does not prove identity of every graph edge. The suite does not constitute a proof over all Git configurations or filesystem layouts.

The baseline cache now separates project-relative roots, architecture identity and analyzer semantics. A payload hash rejects accidentally changed JSON; concurrent writers use distinct temporary publication paths. Neither mechanism authenticates a malicious cache writer. Git inventory and scanning are not an atomic snapshot, so the worktree must remain stable during a check. Ignored source outside excluded build/dependency directories is included because full scanning would read it; the gate must not ignore a relationship merely because Git does.

A.6 Independent follow-up design

The technical extension specifies a future repository frame selected before prediction inspection, with lineage-aware development/pilot/holdout separation. Two independent human reviewers must enumerate supported relations across the complete declared source scope, seal their separate labels, and preserve disagreements before adjudication. Checking emitted predictions alone can estimate precision but cannot establish recall. Supported-vocabulary coverage, execution failures and unresolved labels require separate accounting.

A comparator must share a nonempty, frozen observation unit with ArchSync. An import dependency and a remote service interaction are not interchangeable merely because both can be drawn as graph edges. Performance experiments must pair the same workloads and machines, control the graph-cache condition, separate priming from timed execution and retain failures. Operating-system disk cache is not equivalent to the ArchSync graph cache. These protocols are proposed technical designs, not completed studies or evidence of independent human participation.